Ders 3 Genişletilmiş Notu: Dizi Komutları, BCD Düzenleme, EXE/COM Program Yapısı ve DEBUG Uygulaması

Ders 3 Genişletilmiş Notu: Dizi Komutları, BCD Düzenleme, EXE/COM Program Yapısı ve DEBUG Uygulaması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı ve Paylaşılan Araçlar
- 2. Dizi (String) Komutlarına Giriş
- 3. Dizi Komutları Tek Tek
- 4. İşaret Genişletme: CBW ve CWD
- 5. Tekrar Önekleri: REP, REPE, REPNE
- 6. Uygulamalı Dizi Örnekleri
- 7. Giriş/Çıkış ve İşlemci Denetim Komutları
- 8. BCD Düzenleme Komutları
- 9. EXE ve COM Tipi Program Yapısı
- 10. DOSBox, MASM, LINK ve DEBUG ile Program Çalıştırma: BirSay
- 11. Word Dizide Seçmeli Sıralama: SelSort
- 12. SI ve DI'nin Anlamı, DEBUG ile Bellek İzleme
- Örnek Sorular

1. Dersin Amacı ve Paylaşılan Araçlar

Bu ders, laboratuvar saatinde yapılan online bir ek dersti. Hoca, geçen hafta tamamlanamayan dizi komutlarını bitirmek ve birkaç örnekle bir assembly programının nasıl derlenip çalıştırıldığını göstermek istediğini söyledi. Gerekçe, önümüzdeki haftalarda başlayacak laboratuvar çalışmalarıydı. Öğrencilerin kod yazıp DEBUG ile denemeye bir an önce başlaması gerekiyordu.

Hoca ders duvarında bazı dosyalar paylaştığını anlattı. Bunlardan biri Microsoft'un assembler'ı MASM (Microsoft Macro Assembler) ile EXE üretme ve DEBUG işlemlerini gösteren paketti. İkincisi "boş ASM" adlı, `.asm` uzantılı bir iskelet dosyasıydı. Bu dosyada her programda yazılması gereken standart bölümler hazır bulunuyor: stack segment ve data segment tanımları, code segment içindeki standart satırlar. Öğrenci yalnız code segment içindeki kendi prosedürünü yazarak programı çalıştırabilir.

2. Dizi (String) Komutlarına Giriş

Dizi komutları, bellekte art arda duran karakterleri ya da sayıları alıp işlemek için kullanılan, işi oldukça kolaylaştıran bir komut ailesidir. Bunlara string komutları da denir. Ailede şu komutlar var: `MOVSB/MOVSW` (move string byte/word), `CMPSB/CMPSW` (compare string), `SCASB/SCASW` (scan string), `LODSB/LODSW` (load string), `STOSB/STOSW` (store string). Slaytta kırmızıyla gösterilen tekrar önekleri (`REP`, `REPE/REPZ`, `REPNE/REPNZ`) bu komutlarla birlikte kullanılır.

Dizi komutlarının ortak özellikleri şunlardır:

- Tekrarlı işlemler için tasarlanmışlardır.
- Tekrar sayısı `CX` register'ında tutulur.
- Her işlem adımında `CX` otomatik olarak bir azaltılır.
- Tekrar, komutun önüne yazılan `REP` türü öneklerle belirtilir.

2.1 Yön Bayrağı (Direction Flag)

Dizi işlemlerinde dizinin elemanları bellekten tek tek çekilir. Bu okumanın yönünü, yani adreslerin ileri mi geri mi gideceğini yön bayrağı (direction flag, DF) belirler.

- DF = 0 ise her adımda dizinin bir sonraki adresiyle işlem tekrarlanır. Hocanın örneğiyle, adres FA ise sonraki adım FB olur. Hoca bunu "pozitif yön" gibi düşünmeyi önerdi.
- DF = 1 ise her adımda bir önceki adrese geçilir. Adresler eksilerek gider. Hoca bunu "negatif yön" diye andı.

DF'yi 0 yapmak için `CLD` (clear direction flag) kullanılır. Örneklerde hep bu komut kullanıldı.

> **Ek bilgi (derste anlatılmadı):** DF'yi 1 yapan komut `STD`'dir (set direction flag). Geri yönlü işlem, örneğin bir diziyi sondan başa doğru dolaşmak ya da üst üste binen bellek bölgelerinde kopyalama yapmak için kullanılır.

3. Dizi Komutları Tek Tek

3.1 MOVSB ve MOVSW

`MOVSB`, data segmentte kaynak indisi (source index, `SI`) ile adreslenen byte'ı alır ve ekstra segmentte hedef indisi (destination index, `DI`) ile gösterilen adrese yazar. Kısaca `ES:[DI]` ← `DS:[SI]`.

Hoca burada adreslemeyi hatırlattı. Kodda adresleri hep 16 bit gösteriyoruz. Oysa dersin başında konuşulduğu gibi 8086 işlemcide 1 MB'lık bir bellek vardır ve 20 bit ile adreslenir. Kodda görülen 16 bitlik adres, segment register'ının (kesim register'ı) gösterdiği alan içindeki konumdur. `MOVSB` için kaynak, `DS`'nin gösterdiği alanda `SI` ile belirtilen adrestir.

> **Ek bilgi (derste anlatılmadı):** 8086'da 20 bitlik fiziksel adres, segment değeri 16 ile çarpılıp (bir hex basamak sola kaydırılıp) ofsete eklenerek bulunur: fiziksel adres $=$ segment $\times 16 +$ ofset. `DS:SI` gösterimi bu çifti ifade eder.

Komut çalıştıktan sonra DF'ye bakılır. DF = 0 ise `SI` ve `DI` birer artırılır, böylece bir sonraki adrese geçilir. DF = 1 ise birer azaltılır.

`MOVSW`'de sondaki B (byte) yerine W (word) gelir. Artık bir byte değil, iki byte'lık bir word okunup yazılır. Bellekte art arda gelen iki byte'ı okumak, adresi iki artırmak demektir. Bu yüzden `MOVSW`'de `SI` ve `DI` 2 değişir.

3.2 CMPSB ve CMPSW

`CMPSB`, `SI` ve `DI`'nin gösterdiği iki bellek gözünü karşılaştırır. Hoca, `CMP` komutunu hatırlattı: Karşılaştırma bir çıkarma işlemidir, iki sayının farkı alınır ama sonuç hiçbir yere yazılmaz. Burada da aynı mantık geçerlidir. `DS:[SI] − ES:[DI]` hesaplanır, yalnız bayraklar değişir. Bu bayraklara bakarak iki değerin eşit mi, büyük mü, küçük mü olduğu anlaşılır.

Byte'lık sürümde indisler 1, `CMPSW`'de 2 değişir.

3.3 SCASB ve SCASW

`SCASB`, ekstra segmentte `DI` ile gösterilen adresteki veriyi `AL` ile karşılaştırır. `AL`, hocanın deyişiyle hep üzerinde işlem yaptığımız akümülatördür. Bu komutla aslında bir arama yapılır: `AL`'de tutulan bir karakterin tanımlı bir dizi içinde olup olmadığı bulunur. Eşleşme bulunduğunda, uygun önekle birlikte döngüden çıkılır.

`SCASW` aynı işi 16 bit için yapar. `AL` yerine `AX` kullanılır ve bellekten alınan word `AX` ile karşılaştırılır.

3.4 LODSB ve LODSW

Load işlemi, bellekten okunan bir değeri register'a getirir. LODSB, data segmentte SI ile adreslenen byte'ı AL'ye yükler ve SI'yı DF = 0 ise 1 artırır, DF = 1 ise 1 azaltır. Kısaca AL ← DS:[SI]. Bir karakter katarının içinden bir karakter alınıp AL'ye getirilir. LODSW aynı işi word için yapar.

3.5 STOSB ve STOSW

Store, load'un tersidir. Register'da tutulan veri, bellekte dizi olarak tanımlanmış bir alana yazılır. STOSB, AL'deki byte'ı ES:[DI]'ya yazar. İşlem kaç kez tekrarlanacaksa, örneğin 5 kez, bu sayı CX'e konur.

4. İşaret Genişletme: CBW ve CWD

Hoca bu kısmı "çok önemli, zaman zaman ciddi ihtiyaç duyduğumuz" komutlar diye tanıttı.

> **Hocanın vurgusu:** CBW ve CWD çok dikkat edilmesi gereken komutlardır. Genişletme işaret biti dikkate alınarak yapılır.

CBW (convert byte to word), 8 bitlik bir register'daki veriyi 16 bitlik register'a genişleterek (extend) yazar: AL'den AX'e. Burada dikkat edilecek şey işaret genişletmedir (sign extension). Sayının işaret biti dikkate alınmalıdır.

AL'de negatif bir sayı olduğu nasıl anlaşılır? AL'nin en yüksek anlamlı biti 1 ise sayı negatiftir. Bu negatif sayı 16 bitlik register'a yazılırken işaret biti yüksek anlamlı kısma yayılır. Örneğin AL = F0H ise CBW sonrası AX = FFF0H olur.

CWD (convert word to double word) aynı işi 16 bitten 32 bite yapar. Kullandığı register çifti DX:AX'tir. Hoca, bölme işlemlerinde böyle 32 bitlik bir register yapısına ihtiyaç duyulduğunu hatırlattı. 16 bitlik bölmede bölünen DX:AX'te durduğu için CWD gerekir.

5. Tekrar Önekleri: REP, REPE, REPNE

Dizi komutları REP, REPE (repeat equal) ya da REPNE (repeat not equal) önekleriyle kullanılır.

REP için örnek: CX'e 10 atandığını düşünelim. Bu sayı, dizi işleminin art arda gelen kaç adres için tekrarlanacağını gösterir. REP MOVSB yazıldığında, aynı satırdaki MOVSB işlemi CX'teki sayı kadar tekrarlanır.

REPE "eşitse tekrarla" mantığıyla çalışır. Tekrar, CX sıfır olana kadar ya da sıfır bayrağı (zero flag, ZF) 0 olana kadar sürer. REPNE bunun tersidir: CX sıfır olana kadar ya da ZF 1 olana kadar, yani ilk eşleşmeye kadar tekrarlar.

- REP: CX ≠ 0 olduğu sürece tekrarlar.
- REPE/REPZ: CX ≠ 0 ve ZF = 1 olduğu sürece tekrarlar. İlk farkta durur.
- REPNE/REPNZ: CX ≠ 0 ve ZF = 0 olduğu sürece tekrarlar. İlk eşleşmede durur.

5.1 Slayttaki İki Kısa Örnek

İlk örnekte bellekte iki dizi var: dizi1 değişkeniyle adreslenen bir dizi ve dizi2 adresine yerleştirilmiş bir başka alan. LEA SI, dizi1 (load effective address) ile dizi1'in adresi SI'ya, LEA DI, dizi2 ile dizi2'nin adresi DI'ya alınır. CLD ile DF sıfırlanır, yani artan adreslerle çalışılır. CX'e dizi boyutunu tutan size yüklenir. REP MOVSB ile SI'daki eleman DI'daki alana kopyalanır ve bu CX kadar tekrarlanır. Sonuçta dizi1, dizi2 bölgesine kopyalanmış olur.

```
LEA SI, dizi1
LEA DI, dizi2
CLD
MOV CX, size
REP MOVSB        ; dizi2 <- dizi1
```

İkinci örnek aynı işi STOSB ile yapar. Dizinin adresi DI'ya yazılır, CLD denir ve XOR AL, AL ile AL sıfırlanır. CX'e yine size konur. REP STOSB, AL'deki 0 değerini DI'nın gösterdiği adresten başlayarak dizinin her byte'ına yazar. 1 byte'lık bir operand üzerinde çalışıldığı için STOSB

kullanılır. Hoca dizinin 5 elemanlı olduğunu varsaydı: 5 byte'lık alanın her byte'ına 0 yüklenir. Bu, diziye ilk değer olarak 0 atamaktır.

```
LEA DI, dizi
CLD
XOR AL, AL        ; AL = 0
MOV CX, size
REP STOSB         ; dizinin her byte'ına 0 yazılır
```

6. Uygulamalı Dizi Örnekleri

Hoca bu bölümde tablet üzerinden çizerek örnekler gösterdi. Bağlantı programında sorunlar yaşandığı için akış zaman zaman kesildi.

6.1 Örnek 1: REP MOVSB İzlemesi

`address1` adlı adreste bellekte sırasıyla 41H, 50H, 70H, 4AH değerleri var. Hoca hepsinin onaltılık (hexadecimal) olduğunu göstermek için yanlarına H eklemek gerektiğini belirtti. `address1` `SI`'ya, hedef olacak `address2` `DI`'ya alınır. `CX`'e 4 konur ve `CLD` ile DF = 0 yapılır.

```
LEA SI, address1
LEA DI, address2
MOV CX, 4
CLD
REP MOVSB
```

`MOVSB` ilk adımda `address1`'deki 41H'yi `address2`'ye kopyalar. `REP` `CX`'i bir azaltıp 3 yapar ve işlemi tekrarlar. Bu sırada `SI` ve `DI` da birer artmıştır. Böylece bir sonraki adresteki 50H bir sonraki hedef gözüne kopyalanır. İşlem `CX` sıfırlanana kadar sürer. `address1` = 0000H ve `address2` = 2000H için adım adım durum:

Adım	Kopyalanan	SI sonra	DI sonra	CX sonra
1	41H → 2000H	0001H	2001H	3
2	50H → 2001H	0002H	2002H	2
3	70H → 2002H	0003H	2003H	1
4	4AH → 2003H	0004H	2004H	0

Tablo python ile adım adım doğrulandı.

> **Ders sonrası not:** Hoca 00:34:14'te "CX register'ını bir artırarak" dedi. Doğrusu: `CX` her adımda bir azalır. Hoca hemen öncesinde (00:33:56) doğru olarak "bir eksiltip 3 yapıyor" demişti.
>
> **Ders sonrası not:** Kısa nota göre tablette ikinci satır `LEA SI, address2` yazılmıştı. Doğrusu `LEA DI, address2`. Hoca da anlatırken "D, I'ye aldım" diyor.

6.2 Örnek 2: Parola Karşılaştırma (REPE CMPSB)

`dizi1` ve `dizi2`'de iki parola var ve aynı olup olmadıklarını anlamak istiyoruz. İkisi de 10 karakter uzunluğunda: `dizi1` = "password01", `dizi2` = "pasword123". `dizi1`'in efektif adresi `SI`'ya, `dizi2`'ninki `DI`'ya alınır, uzunluk `CX`'e yazılır ve `REPE CMPSB` çalıştırılır.

```
LEA SI, dizi1
LEA DI, dizi2
MOV CX, 10
REPE CMPSB
```

`REPE` "eşit olduğu sürece karşılaştır" der ve karakterler byte byte kıyaslanır. P uydu, A uydu, üçüncü harf S uydu. Dördüncü harfte "s" ile "w" farklı olduğu için, `CX` sıfırlanmadan döngüden çıkılır.

Python ile doğrulama: Döngü 4 karşılaştırma yapar. Çıkışta $CX = 10 - 4 = 6$ ve ZF = 0 olur.

6.3 Örnek 3: E-posta Adresinde “@” Arama (REPNE SCASB)

Bu örneği hoca aradan hemen önce gösterdi. Amaç, bir e-posta adresinin içinde “@” sembolü olup olmadığını kontrol eden bir kod yazmak. “@” sembolünün ASCII karşılığı 40H’dir, ASCII tablosunda görülebilir. `email` değişkeninin bellekteki adresi 0200H olsun ve 10 byte uzunluğunda olsun: 0200H, 0201H, 0202H diye gider. Bu adreslerden birinde 40H olduğu görülüyor. Kısa nota göre 40H, 0205H adresindedir.

```
LEA DI, email
MOV CX, 10
CLD
MOV AL, '@'
REPNE SCASB      ; AL ile ES:[DI] eşleşene kadar ara
```

`REPNE SCASB`, bu alandaki adresleri tek tek 40H ile eşleşiyor mu diye kontrol eder. Eşleşme yakalanırsa döngüden çıkılır. Bulunamazsa `CX` sıfırlanarak çıkılır. Hoca, sonucun `CX`’e bakarak anlaşılabileceğini söyledi.

Python ile izleme (40H, 0205H’de): Eşleşme 6. karşılaştırmada olur. Çıkışta ZF = 1, `CX` = 4 ve `DI` = 0206H olur. `DI`, eşleşen byte’ın bir ötesini gösterir.

7. Giriş/Çıkış ve İşlemci Denetim Komutları

Hoca giriş/çıkış komutlarının asıl uygulamasının birinci vizeden sonra, çevre birimleri (arayüzler) konusunda yapılacağını söyledi ve kısaca geçti.

Klavye, monitör, fare gibi işlemciye çevre birimi olarak bağlanan bütün cihazlar, işlemciye bir arayüz (interface) üzerinden veri aktarır. Bu arayüzlerin içindeki register’lara port denir. Portları okumak ve yazmak için `IN` ve `OUT` komutları kullanılır.

`IN` komutu, numarası verilen bir porttan okuma yapar. Port adresi anlık veri (immediate data) olarak verilirse 8 bitliktir ve 0-255 arasında değer alır. Örneğin `IN AL, 38H` ile 38H adresindeki porttan okunur. Okunan veri akümülatöre gider. Bu komutlar yalnız `AL` ve `AX` üzerinde çalışır; veri yolu genişliğine göre 8 ya da 16 bit aktarılır.

Anlık port numarasının 0-255 ile sınırlı olması giriş/çıkış uzayını kısıtlar. Normalde daha büyük bir port alanı adreslenebilir. 65536 farklı porta erişmek için port numarası `DX`’e konur.

> **Ders sonrası not:** Altyazıda (00:39:39) “65536 farklı port üzerinden okuma ya da yazma işlemi yapmak istiyorsak port numarasının DX register’ında olması gerekmiyor” geçiyor. Doğrusu: 255’ten büyük port adresleri için port numarası `DX`’te olmalıdır, örneğin `IN AL, DX`.

`OUT` komutu ters yönde çalışır: Akümülatörde tutulan veri bir porta yazılır. Hoca gerekçeyi şöyle anlattı: Klavyeden ya da fareden giriş yaparız, ama ekrana çıkış göndeririz. Porta yazmak istenen veri `OUT` ile gönderilir.

İşlemciyi durdurma ve bekletme komutları da var:

- `HLT`: İşlemciyi durdurur. Bir kesme (interrupt) gelince işlemci kaldığı komuttan devam eder.
- `NOP` (no operation): Hiçbir işlem yapmaz. Amacı gecikmedir (delay). Başka bir kanaldan gelecek verinin hazır olması için zaman gerekiyorsa, kod akışına `NOP` ile gecikme eklenebilir.
- `WAIT`: Yardımcı işlemcinin (coprocessor) sonucunu göndermesi için işlemciyi bekletir.

Altyazıda “alt komutu” diye geçen iki yer bağlama göre `OUT` (00:40:16) ve `HLT` (00:40:33) komutlarıdır.

8. BCD Düzenleme Komutları

İkili kodlanmış ondalık (binary coded decimal, BCD) bir sayı gösterimidir. Normalde onaltılık ya da ikili tuttuğumuz bir sayıyı BCD düzeninde göstermek gerekebilir. Hoca örnek olarak bir LED göstergeye, 7 segment göstergeye (display) veri aktarmayı verdi.

Altı komut var:

- AAA (ASCII adjust after addition)
- AAS (ASCII adjust after subtraction)
- AAM (ASCII adjust after multiplication)
- AAD (ASCII adjust before division)
- DAA (decimal adjust after addition)
- DAS (decimal adjust after subtraction)

8.1 AAA

AAA, ASCII değer olarak tutulan iki sayının toplamını BCD olarak düzenler. Örnekte önce XOR AX, AX ile AX sıfırlanır. MOV AL, '6' ile 6'nın ASCII kodu AL'ye konur, ADD AL, '7' ile 7'nin ASCII koduyla toplanır.

```
XOR AX, AX
MOV AL, '6'       ; 36H
ADD AL, '7'       ; AL = 6DH
AAA               ; AX = 0103H, BCD olarak 13
```

Çözüm adım adım:

- Verilen: AL = 36H ('6'), eklenen 37H ('7').
- Toplama: 36H + 37H = 6DH.
- Ondalık olarak $6 + 7 = 13$ olmalıdır. 13'ü BCD göstermek, AX'in yüksek anlamlı kısmında 1, düşük anlamlı kısmında 3 olması demektir.
- AAA sonrası AX = 0103H. Yani AH = 01, AL = 03.

Python ile doğrulandı. Hoca, ASCII düzenleme komutlarına belki çok ihtiyaç duyulmayacağını, ondalık düzenleme komutlarının daha çok kullanılacağını söyledi.

8.2 AAD

AAD'de yüksek anlamlı kısma 04, düşük anlamlı kısma 05 yüklenir. Hoca değerleri onaltılık yükledi. Ama BCD olarak bakıldığında AX'teki sayının değeri 45'tir.

```
MOV AH, 04H
MOV AL, 05H       ; AX'te BCD 45
AAD               ; AX = 002DH (45 ondalık)
```

BCD olarak 45 görünen bu sayı, bölme gibi bir işlemde kullanılacaksa ikili (onaltılık) karşılığı gerekir. AAD bunu üretir: $4 \times 10 + 5 = 45 = 2$DH. Sonuç AX = 002DH olur. Python ile doğrulandı.

Hoca AAM ve AAS'ı benzer oldukları için hızlı geçti.

8.3 DAA

Hoca, DAA ve DAS'ın daha çok kullanılacak komutlar olduğunu söyledi ve dikkat istedi.

```
MOV AL, 75H
ADD AL, 19H       ; AL = 8EH
DAA               ; AL = 94H (75 + 19 = 94)
```

Çözüm adım adım:

- Verilen: AL = 75H, eklenen 19H.
- Toplama: 75H + 19H = 8EH. Bu onaltılık sonuçtur.
- Değerler BCD sayı gibi düşünülürse ondalık toplam istenir. Hocanın anlatımıyla: $5 + 9 = 14$; 4 yazılır, elde 1. Sonra $7 + 1 + 1 = 9$. Sonuç 94.
- DAA sonrası AL = 94H.

Python ile doğrulandı.

> **Ek bilgi (derste anlatılmadı):** DAA düzeltmeyi şöyle yapar: AL'nin düşük dört biti 9'dan büyükse ya da yardımcı elde bayrağı (AF) 1 ise AL'ye 06H eklenir. Ardından üst dört bit 9'u aşarsa ya da elde bayrağı (CF) 1 ise 60H eklenir. Dersteki örnekte 8EH'nin düşük basamağı EH > 9 olduğu için 8EH + 06H = 94H olur. DAS aynı düzeltmeyi çıkarmayla yapar.

8.4 DAS

```
MOV AL, 42H
SUB AL, 13H      ; AL = 2FH
DAS              ; AL = 29H (42 - 13 = 29)
```

AL'ye 42H yüklenir ve 13H çıkarılır. Sonuç onaltılık olarak 2FH'dir. DAS sonrası AL = 29H olur. Bu, iki BCD sayının farkının ondalık gösterimidir: $42 - 13 = 29$. Python ile doğrulandı.

8.5 Aradan Sonra Gelen DAA Sorusu

Aradan sonra hoca, DAA işlemi için değerlerin tam olarak nereden geldiğini soran bir soruyu yanıtladı. Register'da tutulan sayılar BCD formatlı düşünülürse 75 ile 19'un toplamı 94'tür. MOV AL, 75H ve ADD AL, 19H yapıldıktan hemen sonra DAA çalıştırılır. DAA yalnız akümülatörün düşük anlamlı sekizlisi AL üzerinde ondalık düzenleme yapar. Toplama sonrası onaltılık değer DAA ile 94H'ye dönüştürülür. Hoca bunu bir format dönüşümü olarak özetledi.

> **Ders sonrası not:** Hoca bu açıklamada (01:05:10) toplamın "9E olarak göründüğünü" söyledi. Doğrusu 8EH'dir (75H + 19H = 8EH). Hoca dersin ilk kısmında (00:45:59) doğru değeri söylemişti.

9. EXE ve COM Tipi Program Yapısı

Hoca derleme uygulamasına geçmeden önce bir assembly programının bölümlerini anlatan slaytı açtı. Assembly programları kodlama şekli, kesim (segment) tanımları ve kesim düzenine göre iki biçimde yazılabilir. Derlenen programın uzantısına göre bunlara EXE tipi ve COM tipi program denir.

> **Hocanın vurgusu:** Bu dönem yalnız EXE tipi programlara odaklanılacak. COM tipine yalnız yapısal farkı görmek için değinilecek, onunla uygulama yapılmayacak.

9.1 EXE Tipi Programın Bölümleri

```
STACKSG SEGMENT PARA STACK 'STACK'
        DW 32 DUP(?)
STACKSG ENDS

DATASG  SEGMENT PARA 'DATA'
SAYI    DB ?
ELEMAN  DW ?
DATASG  ENDS

CODESG  SEGMENT PARA 'CODE'
        ASSUME CS:CODESG, DS:DATASG, SS:STACKSG
BASLA   PROC FAR
        PUSH DS
        XOR AX, AX
        PUSH AX
        MOV AX, DATASG
        MOV DS, AX
        ; kendi kodunuz buraya
        RETF
BASLA   ENDP
CODESG  ENDS
        END BASLA
```

Stack segment. Segmentin bir adı ve stack segment olduğunu belirten bir bölümü vardır. Hizalama türü paragraf, byte ya da word olabilir, ama derste hep paragraf (PARA STACK 'STACK') kullanılacak. Paragraf kullanımı, segmentin ana bellekte 16'ya bölünebilen, yani onaltılık son basamağı 0 olan bir adresten başlayacağını gösterir. Hoca bunun şu an çok önemli olmadığını belirtti. DW 32 DUP(?), 32 kez tekrarlanan bir word yapısıdır, yani 32 word'lük bir stack alanı ayrılır.

Data segment. Değişken tanımları burada yapılır. Örneğin `SAYI DB` ile byte tipinde bir değişken tanımlanır. İlk değeri burada verilebilir: 0 ya da başka bir değer. `ELEMAN DW` word tipinde bir başka değişkendir. Segment `ENDS` (end segment) ile kapanır.

Code segment. `CODESG SEGMENT PARA 'CODE'` ile başlar, en alttaki `CODESG ENDS` ile biter. EXE programlarda içeride en az bir tane `FAR` tipi prosedür olmalıdır. Hoca, `FAR` tipinin farklı bir segment adresi verilebilmesi anlamına geldiğini hatırlattı.

Prosedürün başındaki 5 satır, bütün `FAR` prosedürlerde yazılması gereken standart satırlardır:

- İlk üç satır (`PUSH DS`, `XOR AX, AX`, `PUSH AX`), EXE'yi çağıran kodun kesim değerini ve ofset değeri 0'ı dönüş için stack'e yazar. Böylece stack'e art arda iki word konur: dönüş için kesim adresi ve komut göstergesi (instruction pointer) adresi 0.
- Sonraki iki satır (`MOV AX, DATASG`, `MOV DS, AX`), programın kendi data segmentinin adresini `AX` üzerinden `DS`'ye aktarır.

  Hocanın vurgusu: `DS` stack'e doğrudan yazılabilir (push edilebilir). Ama tanımlanan data segmentin adresi `DS`'ye doğrudan yazılamaz; önce `AX`'e alınır, sonra `MOV DS, AX` yapılır. EXE'nin kendi data segmentini `DS` register'ına yazmak gerekir.

Bu beş satırdan sonra kişinin kendi kodu gelir. Kod bitince `RETF` (return far) prosedürün dönüşünü gösterir. Ardından `ENDP` ile prosedür, `ENDS` ile code segment kapanır. En alttaki `END BASLA`, EXE başlatıldığında hangi yordamın çağrılacağını belirtir. Burada `BASLA` ana prosedürdür (main).

9.2 COM Tipi Programın Yapısı

```
CODESG  SEGMENT PARA 'CODE'
        ORG 100H
        ASSUME CS:CODESG, SS:CODESG, DS:CODESG, ES:CODESG
BASLA   PROC NEAR
        ; kod
        RET
BASLA   ENDP
SAYIB   DB ?
SAYIW   DW ?
CODESG  ENDS
        END BASLA
```

COM tipi programlarda yalnız bir segment tanımı vardır: code segment. Data, stack ve code aynı segmentin içinde kullanılır. EXE'de ayrı stack, data ve code segment alanları vardı. Tek segment kullanıldığı için COM programda en az bir `NEAR` prosedür bulunmalıdır; prosedür aynı segmentin içinde tanımlanır. Değişken tanımları da code segmentin içinde yapılır. Sondaki `END` satırı yine COM çalıştırıldığında çağrılacak prosedürü belirtir.

Hoca slayttaki COM-EXE karşılaştırma tablosuna da kısaca değindi. Tabloda kullanılabilir bellek için 64 KB sınırı ve EXE'deki başlık (header) gibi satırlar vardı.

10. DOSBox, MASM, LINK ve DEBUG ile Program Çalıştırma: BirSay

Hoca öğrencilere DOSBox üzerinde assembly kodu çalıştırıp çalıştırmadıklarını, DEBUG ile izleme yapıp yapmadıklarını sordu ve birlikte yapmayı önerdi. Paylaşılan Microsoft ASM paketinin içinden gereken dosyaları bir klasöre koymuştu.

10.1 Kod

Örnek, önceki derste sınıfta çözülen, bir byte içindeki 1'leri sayan programdı. Önceki derste bu iş elde ile döndürme (rotate through carry) komutuyla yapılmıştı. Bu kez kaydırma (shift) ile yapıldı.

```
stacksg SEGMENT PARA 'yigin'
        DW 15 DUP(?)
stacksg ENDS
```

```
datasg  SEGMENT PARA 'veri'
sayi    DB 55H
datasg  ENDS

codesg  SEGMENT PARA 'kod'
        ASSUME CS:codesg, DS:datasg, SS:stacksg
SAY     PROC FAR
        PUSH DS
        XOR AX,AX
        PUSH AX
        MOV AX,datasg
        MOV DS,AX

        XOR BL,BL       ; Reset BL
        MOV CX,8
        MOV AH,sayi
L1:     SHL AH,1
        JNC SON
        INC BL
SON:    LOOP L1
        RETF
SAY     ENDP
codesg  ENDS
        END SAY
```

Hocanın anlatımıyla kodun akışı:

- EXE tipinde yazıldığı için stack segment tanımlandı. Stack üzerinde çok işlem yapılmayacağı için alan 15 word seçildi.
- Data segmentte `sayi` adlı, içindeki 1'ler sayılacak byte tipindeki değişken tanımlandı ve ilk değeri 55H verildi.
- `SAY` adlı prosedürün ilk 5 satırı standart satırlardır.
- `BL`, 1'lerin sayısını tutacak register seçildi ve `XOR BL, BL` ile sıfırlandı.
- Bir byte'lık sayı bit bit kontrol edileceği için `CX`'e 8 kondu.
- 1 byte'lık işlem yapıldığı için `sayi`, `AX`'in yüksek anlamlı kısmı `AH`'ye alındı.
- `SHL AH, 1` ile `AH` bir bit sola kaydırıldı ve dışarı çıkan bit elde bayrağına (carry flag, CF) düştü.
- Elde 1 değilse (`JNC`) `SON` etiketine gidilir. Elde varsa `INC BL` ile sayaç artar. `LOOP L1` işlemi tekrarlar.

Hoca kodu Notepad++'ta yazdığını ve "Language" menüsünden Assembly seçilirse segment tanımlarının ve komutların renklendirilerek düzenli görüldüğünü söyledi. Ayrıca MASM'ın büyük/küçük harf duyarlı (case sensitive) olmadığını belirtti.

10.2 Derleme ve Bağlama

DOSBox içinde sırasıyla:

1. `masm birsay.asm;` ile assembler çalıştırılır ve `.obj` dosyası üretilir. Çıktıda "0 warnings, 0 severe errors" görüldü, yani hata yoktu.
2. `link birsay.obj;` ile bağlayıcı çalıştırılır ve `.exe` dosyası üretilir. Komutlar sonda noktalı virgülle yazıldı.
3. `debug birsay.exe` ile program DEBUG'da açılır.

Link aşamasında bir uyarı çıktı. Hoca bunun önemli olmadığını söyledi.

> **Ders sonrası not:** Kısa nota göre bu uyarı `warning L4021: no stack segment` idi. Nedeni, stack segment tanımında `STACK` birleştirme türünün olmamasıdır (`SEGMENT PARA 'yigin'`). Slayttaki gibi `SEGMENT PARA STACK 'STACK'` yazılırsa uyarı büyük olasılıkla çıkmaz ve `SS:SP` programın kendi stack'ini gösterir.

10.3 DEBUG ile İzleme

DEBUG'da `u` (unassemble) komutu makine kodunu komutlara çevirip gösterir. Hoca, sağ tarafta makine kodunun anımsatıcı (mnemonic) karşılıklarının görüldüğünü anlattı. İlk 5 komut EXE'nin data segment düzenlemesi için gereken standart satırlardır. Sonrasında öğrencinin kendi satırları gelir: `XOR BL, BL`, `MOV CX, 8`, `MOV AH, sayi` ve `SHL AH, 1`.

Her satır `t` (trace) komutuyla tek tek çalıştırılıp register içerikleri izlenir. İlk 5 satır standart olduğu için geçildi.

- `XOR BL, BL`: `BL`, `BX`'in düşük anlamlı kısmında görülür ve sıfırlanır.
- `MOV CX, 8`: `CX`'te önceden 4 görünüyordu. Komut çalışınca 8 yüklendi.
- `MOV AH, [0000]`: Komuttan önce `AX` = 076CH idi. Hoca `AH`'nin kaç olduğunu sordu; bir öğrenci 07 yanıtını verdi. Komut çalışınca `sayi`'nin değeri 55H `AH`'ye yüklendi ve `AX` = 556CH oldu.
- `SHL AH, 1`: 55H = $0101\,0101_2$. Sola kaydırılınca `AH` = AAH olur ve `AX` = AA6CH görülür. Bir öğrencinin belirttiği gibi sola kaydırma 2 ile çarpmadır: $55\text{H} \times 2 = \text{AAH}$. 55H'nin en yüksek anlamlı biti 0 olduğu için dışarı atılan bit 0'dır, CF = 0. Sayaç artmaz, `BX` hâlâ 0'dır.
- Döngü bir kez daha çalıştırıldığında AAH'nin en yüksek biti 1 olduğu için `INC BL` çalıştı ve `BL` bir arttı.

Python ile doğrulandı: İlk kaydırmada `AH` = AAH ve CF = 0. Sekiz adım sonunda `BL` = 4 olur, çünkü 55H'de dört tane 1 vardır.

Hoca `AX`'in neden 556CH olduğunu bir kez daha açıkladı. `MOV AH, sayi` ile adres yüklenmez. `sayi` değişkeninin gösterdiği bellek gözündeki değer, yani 55H, `AH`'ye yüklenir. `AL`'deki 6CH değişmeden kalır.

> **Ders sonrası not:** Hoca "Q ile programı tamamlattığımızda işlemin sonuna kadar çalıştırılmasını sağlamış olduk" dedi. DEBUG'da `q` çıkış (quit) komutudur. Programı sonuna kadar çalıştırmak için `g` (go) kullanılır.

DEBUG'da `t` ile kod satır satır çalıştırılır, `r` ile register içerikleri gösterilir. Bellekte değişiklik yapan bir program izleniyorsa bellek içeriği de görüntülenebilir.

11. Word Dizide Seçmeli Sıralama: SelSort

İkinci ve daha karmaşık örnek, seçmeli sıralama (selection sort) algoritmasının assembly ile yazılmasıydı. Amaç, word tipinde tanımlı, `n` uzunluklu bir diziyi sıralamak. Dizinin her elemanı 16 bitlik bir sayıdır.

11.1 Akış Diyagramı

Hoca bir akış diyagramı paylaştı. `n` dizinin uzunluğu, `dizi` dizinin elemanlarıdır. Akış şöyledir:

- `i = 0`'dan `n - 2`'ye kadar dön. `min = i` al.
  - `j = i + 1`'den `n - 1`'e kadar dön.
    - `dizi[j] < dizi[min]` ise `min = j`. Değilse hiçbir şey yapma.
  - İç döngü bitince `min`, dizinin kalanındaki en küçük elemanın indisidir. `dizi[i]` ile `dizi[min]` yer değiştirir.

Dış döngü neden `n - 2`'ye kadar gider? İç döngü son elemanla karşılaştırmayı zaten yapar. Hoca, üçüncü sınıf öğrencilerinin seçmeli sıralamayı bildiğini varsayarak algoritmanın kendisinden çok değişkenlerin nasıl tanımlandığına odaklandı.

11.2 Register Planı

Assembly'de bir derleyicinin sunduğu kolaylıklar yoktur. Her değişkenin hangi register'da tutulacağı baştan planlanmalıdır.

> **Hocanın vurgusu:** Assembly'de program yazarken kullanılan her değişkenin hangi register'da tutulacağı baştan planlanmalıdır.

- Döngü indisleri için indis register'ları kullanılır: `i` için `SI`, `j` için `DI`.

- LOOP komutu CX'i kullandığı için döngü sayısı CX'te tutulur.
- En küçük elemanın indisi BX'te tutulur.
- Yer değiştirme için 8086'nın hazır XCHG (exchange) komutu kullanılır.

11.3 Kod

```
STEK    SEGMENT PARA STACK 'STK'
        DW 30 DUP(?)
STEK    ENDS

DSG     SEGMENT PARA 'DTS'
n       DW 10
Dizi    DW 11,2,80,20,25,1,3,7,8,9
DSG     ENDS

CSG     SEGMENT PARA 'CODE'
        ASSUME CS:CSG, DS:DSG, SS:STEK
BASLA   PROC FAR
        PUSH DS
        XOR AX,AX
        PUSH AX
        MOV AX, DSG
        MOV DS, AX
;kod buradan sonra
        XOR SI,SI
        MOV CX,n
        DEC CX              ; dış döngü n-1 kez
disDon: MOV BX,SI           ; min = i
        PUSH CX             ; dış sayacı sakla
        MOV DI,SI
        ADD DI,2            ; j = i+1 (word: +2 byte)
        MOV DX,DI
        SHR DX,1            ; DX = j'nin eleman indeksi
        MOV CX,n
        SUB CX,DX           ; iç döngü n-(i+1) kez
icDon:  MOV AX,dizi[DI]
        CMP AX,dizi[BX]
        JGE falseK
        MOV BX,DI           ; min = j
falseK: ADD DI,2
        LOOP icDon
        POP CX
        MOV AX, dizi[SI]    ; XCHG bellek-bellek çalışmaz
        XCHG AX, dizi[BX]
        MOV dizi[SI],AX
        ADD SI,2
        LOOP disDon
        RETF
BASLA   ENDP
CSG     ENDS
        END BASLA
```

Kod, kısa nottaki ekran okumasından alınmıştır. Kısa nota göre son satırlar ekranda tam görünmüyordu ve akış diyagramına göre tamamlanmıştı. MASM büyük/küçük harf duyarlı olmadığı için Dizi ve dizi aynı etikettir.

11.4 Tanımlar Üzerine Notlar

Hoca stack için 30 word ayrıldığını, bunun aslında gereksiz büyük bir alan olduğunu söyledi. Küçük örnekler için 15 civarı yeterlidir; "elimiz alışmış, büyük yazıyoruz" dedi.

> **Hocanın vurgusu:** Çok büyük stack alanı açmamak doğru olur.

`n`, örnekte 10 alındı. Ama 500 uzunluklu bir diziyle de çalışmak istenebilir. Bu yüzden `n DW` ile tanımlandı. Dizi elemanları da 255'ten büyük sayılar olabileceği düşünülerek `DW` ile tanımlandı.

11.5 Kodun Adım Adım Açıklaması

Dış döngü hazırlığı. `XOR SI, SI` ile `i` sıfırlanır. `CX`'e `n` yüklenir ve bir azaltılır, çünkü dış döngüde son eleman için işlem yapılmayacaktır.

Dış döngü başı. `SI`'nın gösterdiği konum `min` olarak `BX`'e alınır. Ardından iç döngü açılacaktır. İç döngü de `LOOP` ile `CX`'i kullanacağı için dış döngünün `CX` değeri stack'e kaydedilir (`PUSH CX`).

j'nin başlangıcı. `SI`'nın değeri `DI`'ya atılır ve `DI` 2 artırılır. Böylece `DI`, `j = i + 1` gözünü gösterir. Neden 1 değil de 2? Dizideki her eleman word uzunluğundadır, bellek ise byte adreslemelidir. Bir byte bir adreste tutulur; iki byte'lık bir veri iki adres yer kaplar. `ADD DI, 2` ile `i + 1` indisli word'ün konumu `DI`'da oluşur.

> **Hocanın vurgusu:** Bellek byte adreslemelidir. Word dizide bir sonraki elemana geçmek için indis 2 artırılır.

İç döngü sayısının hesabı. Yüksek seviyeli dillerden farklı olarak, tekrar sayısını, yani `CX`'e konacak sayıyı programcı hesaplamalıdır. `j`, `i + 1`'den başladı ve `n − 1`'e kadar gidecek. `DX = DI` alınır. `DX` bir byte ofseti gösterdiği için `SHR DX, 1` ile bir bit sağa kaydırılarak ikiye bölünür. Böylece bu konumun kaçıncı word elemanı olduğu bulunur. Sonra `CX`'e `n` yüklenir ve bu eleman sayısı çıkarılır. `CX` artık iç döngünün kaç kez döneceğini tutar: $n - (i + 1)$.

İç döngü gövdesi. `dizi[DI]`, yani `j` indisli göz, `AX`'e alınır. `BX`'teki en küçük elemanın gözüyle karşılaştırılır. `dizi[j]` büyük ya da eşitse `falseK` koluna gidilir. Değilse `MOV BX, DI` ile `min = j` yapılır.

Ortak kol. `falseK`'da `DI` 2 artırılır; `j` her seferinde bir sonraki word'ü gösterir. `LOOP icDon` bu işlemleri `CX`'teki sayı kadar tekrarlar.

Yer değiştirme. İç döngüden çıkınca dış döngünün `CX` değeri stack'ten geri çekilir (`POP CX`). Register sayısı az olduğu için `CX` daha önce stack'e konmuştu. Sonra yer değiştirme yapılır.

> **Hocanın vurgusu:** `XCHG` iki bellek gözü arasında çalışmaz. Değerlerden biri önce bir register'a alınmalıdır.

Bu yüzden `dizi[SI]` önce `AX`'e alınır, `XCHG AX, dizi[BX]` ile `AX` ve `dizi[BX]` yer değiştirir, son olarak `AX`'teki değer `dizi[SI]`'ya yazılır. Bir yer değiştirme böylece üç satırda yapılmış olur.

Dış döngü sonu. `SI` word tipinde değişken tuttuğu için 2 artırılır ve dış döngü etiketine (`disDon`) dönülür. Kalan satırlar ana prosedürden çıkış, code segmentin kapanışı ve EXE çalıştırıldığında prosedürün başlamasını sağlayan satırlardır.

11.6 Doğrulama

Kod, register düzeyinde aynı adımlarla python'da çalıştırıldı. Her dış turdan sonra dizi:

Dış tur	Dizi
Başlangıç	11, 2, 80, 20, 25, 1, 3, 7, 8, 9
1	1, 2, 80, 20, 25, 11, 3, 7, 8, 9
2	1, 2, 80, 20, 25, 11, 3, 7, 8, 9
3	1, 2, 3, 20, 25, 11, 80, 7, 8, 9
4	1, 2, 3, 7, 25, 11, 80, 20, 8, 9
5	1, 2, 3, 7, 8, 11, 80, 20, 25, 9
6	1, 2, 3, 7, 8, 9, 80, 20, 25, 11
7	1, 2, 3, 7, 8, 9, 11, 20, 25, 80
8-9	1, 2, 3, 7, 8, 9, 11, 20, 25, 80

Sonuç: `1, 2, 3, 7, 8, 9, 11, 20, 25, 80`. Hocanın söylediği gibi her dış turda kalan kısmın en küçük elemanı dizinin başına doğru yerleşir.

JGE işaretli karşılaştırmadır (jump if greater or equal). Dizide negatif sayı olsa da sıralama doğru olur.

12. SI ve DI'nin Anlamı, DEBUG ile Bellek İzleme

Hoca, öğrencilerin kodu ders duvarından alıp DOSBox'ta MASM ile derlemelerini, DEBUG'da adım adım çalıştırıp değişkenlerin aldığı değerleri izlemelerini tavsiye etti. Bu pratik kazanılırsa laboratuvarda daha rahat edilecekti.

Hoca özellikle şu noktayı vurguladı: Bu örnekte SI ve DI adres değil, dizinin indisini tutar. Bunlara LEA ile adres yüklenmemiştir. Böyle bakılırsa kod daha hızlı kavranır.

> **Ders sonrası not:** Daha kesin söylemek gerekirse SI ve DI burada dizi etiketine göre byte ofsetini tutar. dizi[SI] yazımında adres dizi + SI olarak hesaplanır. Bu yüzden word dizide 2'şer artırılırlar. Eleman indisi SHR DX, 1 ile ikiye bölünerek bulunur.

Hoca, assembly dersinin aslında zor bir ders olmadığını söyledi. Kod yazıp sonucunu adım adım görerek ilerlenirse yapılan hatalar da daha iyi anlaşılır. DEBUG ile her register'ın ve belleğin nasıl değiştiği görülebilir.

Sıralamanın bellekte nasıl ilerlediğini görmek için data segmentin görüntülenmesi gerekir. Dizi elemanları bellekte durur. Küçükten büyüğe sıralamada en küçük sayı dizinin başına yerleşir; ikinci turda ikinci en küçük sayı, dizi değişkeninin bellek gözlerinde yerini alır. Bunu görmek için DEBUG'da görüntüleme (display/dump) anlamına gelen d komutu kullanılır: d ds:0 yazılırsa data segmentin 0 adresinden başlayarak yerleştirilen değişkenler görülür.

DEBUG komutlarının özeti:

- u: Makine kodunu komutlara çevirip gösterir (unassemble).
- t: Tek adım çalıştırır (trace).
- r: Register'ları gösterir.
- d ds:0: Data segmentin 0 adresinden belleği gösterir (dump).
- g: Programı sonuna kadar çalıştırır.
- q: DEBUG'dan çıkar.

12.1 Ekstra Segment Üzerine Düzeltme

Hoca CMPSB'yi anlatırken (00:19:28) ekstra segment belirtilmediyse data segment olarak bulunduğunu söyledi.

> **Ders sonrası not:** Bu doğru değil. ES otomatik olarak DS'ye eşitlenmez. Kısa nota göre DEBUG ekranında da DS = 076C ve ES = 075A görünüyordu. Dizi komutları hedefi her zaman ES:DI'dan alır. EXE programda bu komutları kullanmadan önce ES de data segmente yüklenmelidir: MOV AX, DATASG, MOV DS, AX satırlarına ek olarak MOV ES, AX.

Örnek Sorular

Bu kayıt için hazır kare ya da video yoktu. Sorular derste sözlü sorulanlardan alınmıştır.

Soru 1 (derste sözlü, 00:36:56). REPE CMPSB döngüsünden çıkınca iki dizinin aynı olup olmadığını anlamak için CX mi kontrol edilir?

Hoca evet dedi: Çıkışta CX kontrol edilmelidir.

> **Ders sonrası not:** CX kontrolü tek başına yetmez. Fark son karakterde çıkarsa CX yine 0 olur ama diziler farklıdır. Güvenli kontrol ZF ile yapılır: REPE CMPSB sonrası JE (ZF = 1) eşit, JNE (ZF = 0) farklı demektir. Parola örneğinde çıkışta CX = 6 ve ZF = 0'dır.

Soru 2 (derste sözlü, 01:04:01). DAA işleminde düzeltme tam olarak hangi değerlere göre yapılıyor?

Cevap: DAA, hemen önceki toplama işleminin AL'de bıraktığı sonuç üzerinde çalışır. MOV AL, 75H ve ADD AL, 19H sonrası AL = 8EH olur. Register'daki sayılar BCD gibi düşünülürse doğru sonuç 94'tür. DAA yalnız AL üzerinde düzeltme yaparak AL'yi 94H'ye çevirir.

Soru 3 (derste sözlü, 01:26:24). DEBUG ekranında AX = 076CH görünüyor. AH'nin değeri nedir?

Cevap: AH, AX'in yüksek anlamlı byte'ıdır: AH = 07H. Bir öğrenci doğru yanıt verdi. MOV AH, sayi çalıştıktan sonra AX = 556CH olur.

Soru 4 (derste sözlü, 01:27:30). AH = 55H iken SHL AH, 1 çalışırsa AH ne olur?

Cevap: 55H = $0101\,0101_2$. Sola bir kaydırma: $1010\,1010_2$ = AAH. Bir öğrencinin dediği gibi sola kaydırma 2 ile çarpmadır: $85 \times 2 = 170 =$ AAH. Python ile doğrulandı.

Soru 5 (derste sözlü, 01:28:25). Bu kaydırmada elde bayrağına hangi bit atılır, sayaç artar mı?

Cevap: 55H'nin en yüksek anlamlı biti 0'dır. Dışarı atılan bit 0 olduğu için CF = 0 olur, JNC SON dallanır ve BL artmaz.

Soru 6 (derste sözlü, 01:34:58). Seçmeli sıralamada dış döngü neden n − 1'e kadar değil de n − 2'ye kadar döner?

Cevap: İç döngü j = i + 1'den n − 1'e kadar gider ve son elemanla karşılaştırmayı zaten yapar. i = n − 2 turunda son iki eleman karşılaştırılıp yerleşir. Tek elemanlı kalan son kısım için ayrı bir tur gerekmez. Kodda bu yüzden CX = $n - 1$ alınır, yani dış döngü $n - 1$ kez döner.

Dersin Toparlaması

Bu derste 8086'nın dizi komutları tamamlandı. MOVS, CMPS, SCAS, LODS ve STOS komutlarının kaynağı DS:SI'dan, hedefi ES:DI'dan aldığı, byte sürümlerinde indislerin 1, word sürümlerinde 2 değiştiği ve yönün DF ile belirlendiği görüldü. REP, REPE ve REPNE önekleriyle bu komutların CX kadar ya da bir eşitlik/farklılık koşuluna kadar tekrarlandığı; kopyalama, parola karşılaştırma ve karakter arama örnekleriyle izlendi. İşaret genişletme (CBW, CWD), giriş/çıkış komutları ve BCD düzenleme komutları (AAA, AAD, DAA, DAS) slayt örnekleriyle anlatıldı.

Dersin ikinci yarısında EXE ve COM program yapıları karşılaştırıldı ve EXE iskeletindeki standart 5 satırın anlamı açıklandı. Ardından 1'leri sayan program DOSBox'ta MASM ve LINK ile derlendi, DEBUG'da t ile adım adım çalıştırılarak register değişimleri izlendi. Son olarak word dizide seçmeli sıralamanın assembly kodu register planı, iç döngü sayısının hesabı ve XCHG kısıtıyla birlikte anlatıldı. Hoca ertesi günkü yüz yüze derste bu kodu DEBUG'da çalıştırıp dizinin bellekte nasıl değiştiğini data segment üzerinden göstereceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.